

Technical Evaluation Report

JAVA Assessment

Candidate:	nitin.bharti@irissoftware.com
Email:	nitin.bharti@irissoftware.com
Tech Stack:	JAVA
Completed:	August 19, 2026 at 04:27 PM
Report Generated:	August 19, 2026 at 04:27 PM

Overall Assessment

10.0 / 100

Evaluation Duration

Metric	Value
Actual Time Taken	19.6 minutes
Planned Duration	30 minutes
Time Efficiency	65.3% of planned time ✓

Category Scores

Category	Score
Fundamentals & Architecture	9.0/100
Kafka Producer & Consumer	0.0/100
Semantics & Kafka connect	24.0/100
Terminologies	18.0/100
Distributed messaging system	0.0/100

Communication Assessment

Metric	Score
Confidence Level	0/10
English Proficiency	0/10
Clarity Score	0/10

Strengths

- Uses the correct high-level transactional flow: initialize, begin, send two records, commit
- Uses Kafka AdminClient API in a generally appropriate way for programmatic topic deletion
- Includes bootstrap server, entity type, entity name, and describe flags
- Uses the right general Kafka admin area for topic configuration inspection
- Waits for the delete operation to complete with `result.all().get()`

Areas for Improvement

- Candidate did not provide an answer
- Provide a shell command only if the question asks for CLI; do not present it as Java code
- If submitting Java anyway, use a valid entry point: `public static void main(String[] args)`
- Ensure the solution is complete with necessary imports and correct class usage
- Fix Kafka configuration property names to valid keys

Detailed Question Feedback

Q1: Write the Kafka CLI command to permanently delete a topic named "test-topic".

Category: Fundamentals & Architecture | Question Category: Kafka_Upskilling | Difficulty: easy | Type: CODE

Candidate's Code Answer:

```
public class DeleteKafkaTopic{
    private static void main(String args[]){
        Properties config=new Properties();
        config.put(AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");

        try(AdminClient adminClient= AdminClient.create(config)){
            DeleteTopicResult result=adminClient.deleteTopics(Collections.singleton("test-topic"));
            result.all().get();
        }
        catch(Exception e){
            System.err.println("Error Occurred");
        }
    }
}
```

```
}  
}  
}
```

Score: 18/100

Good: Uses Kafka AdminClient API in a generally appropriate way for programmatic topic deletion, Uses try-with-resources to close the AdminClient, Waits for the delete operation to complete with `result.all().get()`

Improve: Answer the actual question with a Kafka CLI command instead of Java code, If submitting Java anyway, use a valid entry point: `public static void main(String[] args)`, Avoid hardcoding the bootstrap server unless the prompt specifies it, Handle specific exceptions more precisely instead of catching generic `Exception`, Ensure the solution is complete with necessary imports and correct class usage

Detailed Feedback:

This does not correctly answer the question asked. The prompt explicitly asks for the Kafka CLI command to permanently delete a topic named "test-topic", but the submission provides Java code using the Kafka AdminClient API instead of a CLI command. That is a fundamental mismatch with the requirement. Even as Java code, it has issues: the entry point is declared as `private static void main(String args[])`, which is not a valid runnable Java application entry point because `main` must be `public static void main(String[] args)`. The code also omits required imports and depends on Kafka client classes being available. Functionally, the AdminClient deletion call is conceptually valid for deleting a topic, but it still does not satisfy the requested output format. The correct answer should have been a CLI command such as: `kafka-topics.sh --delete --topic test-topic --bootstrap-server localhost:9092`. If using older ZooKeeper-based setups, a legacy variant may exist, but for modern Kafka the bootstrap-server form is the expected answer. ■■■ Note: Code did not compile in the execution environment. This may be due to missing framework dependencies (e.g. Spring, JUnit) not available in the sandbox runner. Code logic was evaluated by AI review. Compiler output: Note: Annotation processing is enabled because one or more processors were found on the class path. A future release of javac may disable annotation processing unless at least one processor is specified by name (-processor), or a search path is specified (--processor-path, --processor-module-path), or annotation processing is enabled explicitly (-proc:only, -proc:full). Use -Xlint:-options to suppress this message. Use -proc:none to disable annotation processing. DeleteKafkaTopic.java:3: error: cannot find symbol Properties config=new Properties(); ^ symbol: class Properties location: class DeleteKafkaTopic DeleteKafkaTopic.java:3: error: cannot find symbol Properties config=new Properties(); ^ symbol: class Properties location: class DeleteKafkaTopic DeleteKafkaTopic.java:4: error: cannot find symbol config.put(AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092"); ^ symbol: variable AdminClientConfig location: class DeleteKafkaTopic DeleteKafkaTopic.java:6: error: cannot find symbol try(AdminClient adminCLient= AdminClient.create(config)){ ^ symbol: class AdminClient location: class DeleteKafkaTopic DeleteKafkaTopic.java:6: error: cannot find symbol try(AdminClient adminCLient= AdminClient.create(config)){ ^ symbol: variable AdminClient location: class DeleteKafkaTopic DeleteKafkaTopic.java:7: error: cannot find symbol DeleteTopicResult result=adminCLient.deleteTopics(Collections.singleton("test-topic")); ^ symbol: class DeleteTopicResult location: class DeleteKafkaTopic DeleteKafkaTopic.java:7: error: cannot find symbol DeleteTopicResult result=adminCLient.deleteTopics(Collections.singleton("test-topic")); ^ symbol: variable Collections location: class DeleteKafkaTopic 7 errors

Q2: Write Java code showing a producer callback that logs success (offset) or failure (exception) asynchronously.

Category: Kafka Producer & Consumer | Question Category: Kafka_Upskilling | Difficulty: hard | Type: CODE

Score: 0/100

Improve: Candidate did not provide an answer

Detailed Feedback:

No feedback available

Q3: Write Java code that begins a transaction, sends two records to different topics, and commits the transaction.

Category: Semantics & Kafka connect | Question Category: Kafka_Upskilling | Difficulty: medium | Type: CODE

Candidate's Code Answer:

```
public class KafkaTransaction{  
    public static void main(String args[]){  
        Properties p=new Properties();
```

```

p.put("bootstrap.server", "localhost:9092");
p.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");
p.put("value.serializer""org.apache.kafka.common.serialization.StringSerializer");
p.put("transaction.id", "my-txn");

try(KafkaProducer<String,String> producer=new KafkaProducer<>(p)){
producer.initTransaction();
producer.beginTransaction();
producer.send(new ProducerRecord<>("topic-a", "key1", "value2"));
producer.send(new ProducerRecord<>("topic-b", "key2", "value2"));
producer.commitTransaction();

}
catch(Exception e){
System.err.println("Exception Occured");
}

}
}

```

Score: 24/100

Good: Uses the correct high-level transactional flow: initialize, begin, send two records, commit, Sends records to two different topics as required, Uses try-with-resources for producer lifecycle management

Improve: Fix Kafka configuration property names to valid keys, Fix the syntax error in the `value.serializer` property assignment, Use the correct Kafka transactional API method name `initTransactions()`, Abort the transaction on failure after beginning the transaction, Handle transactional exceptions more precisely instead of catching generic `Exception` and suppressing details

Detailed Feedback:

This is implementation code, not a test. The intent is correct: create a Kafka producer, initialize transactions, begin a transaction, send two records to different topics, and commit. However, the submission is not technically correct Java/Kafka code as written and would not compile or run successfully without fixes. There are multiple API/property name errors: `bootstrap.server` should be `bootstrap.servers`, `transaction.id` should be `transactional.id`, `initTransaction()` should be `initTransactions()`, and the `value.serializer` line is syntactically broken because it is missing a comma between key and value. Because of these issues, the code does not correctly solve the problem in executable form. Error handling is also weak for transactional Kafka usage: on failure after `beginTransaction()`, the producer should typically call `abortTransaction()` rather than just printing an error. Catching generic `Exception` and swallowing details is also poor practice. The correct approach is to configure a transactional producer with valid Kafka properties, call `initTransactions()`, `beginTransaction()`, send both records, then `commitTransaction()`, and abort on transactional failure. ■■■ Note: Code did not compile in the execution environment. This may be due to missing framework dependencies (e.g. Spring, JUnit) not available in the sandbox runner. Code logic was evaluated by AI review. Compiler output: Note: Annotation processing is enabled because one or more processors were found on the class path. A future release of javac may disable annotation processing unless at least one processor is specified by name (-processor), or a search path is specified (--processor-path, --processor-module-path), or annotation processing is enabled explicitly (-proc:only, -proc:full). Use -Xlint:-options to suppress this message. Use -proc:none to disable annotation processing. KafkaTransaction.java:6: error: `)' or `,' expected
p.put("value.serializer""org.apache.kafka.common.serialization.StringSerializer"); ^ 1 error

Q4: Write the Kafka CLI command to view the retention.ms setting configured for topic "orders".

Category: Terminologies | Question Category: Kafka_Upskilling | Difficulty: medium | Type: CODE

Candidate's Code Answer:

```

kafka-config.sh --bootstrap-server localhost:9092 --entity-type topics --entity-name orders
--describe

```

Score: 18/100

Good: Uses the right general Kafka admin area for topic configuration inspection, Includes bootstrap server, entity type, entity name, and describe flags

Improve: Use the correct Kafka CLI script name: `kafka-configs.sh`, Answer the specific requirement to view `retention.ms` rather than all configs generically, Provide a shell command only if the question asks for CLI; do not present it as Java code, Ensure the command is complete and precise for the requested setting

Detailed Feedback:

This does not correctly solve the question as written. The prompt asks for the Kafka CLI command to view the `retention.ms` setting for topic "orders". The submitted command is a shell command, not Java, and it is also incomplete for the specific ask because it only describes all topic-level configs for the topic. It does not specifically filter or target `retention.ms`. In practice, ``kafka-configs.sh --describe`` can show the topic configs, and `retention.ms` may appear if explicitly set, but this is not a precise answer to "view the `retention.ms` setting configured". Also, the script name is wrong for standard Kafka distributions: it is typically ``kafka-configs.sh``, not ``kafka-config.sh``. A correct CLI-oriented answer would be something like ``kafka-configs.sh --bootstrap-server localhost:9092 --entity-type topics --entity-name orders --describe | grep retention.ms`` if the goal is to view that specific setting, or simply the correct ``kafka-configs.sh ... --describe`` command if filtering is not required. As submitted, it is technically weak and not complete. ■■ Note: Compilation failed due to environment/infrastructure issues (e.g. missing JUnit/Spring jars in the runner). No score penalty applied. Compiler output: Could not find a public class in the code

Q5: Complete the Python consumer snippet to subscribe to "user-events" and print each message's key and value.

Category: Distributed messaging system | Question Category: Kafka_Upskilling | Difficulty: medium | Type: CODE

Score: 0/100

Improve: Candidate did not provide an answer

Detailed Feedback:

No feedback available

Q6: Complete the Python snippet to fetch and print the cluster's broker list using `confluent_kafka's` `AdminClient`.

Category: Fundamentals & Architecture | Question Category: Kafka_Upskilling | Difficulty: medium | Type: CODE

Score: 0/100

Improve: Candidate did not provide an answer

Detailed Feedback:

No feedback available